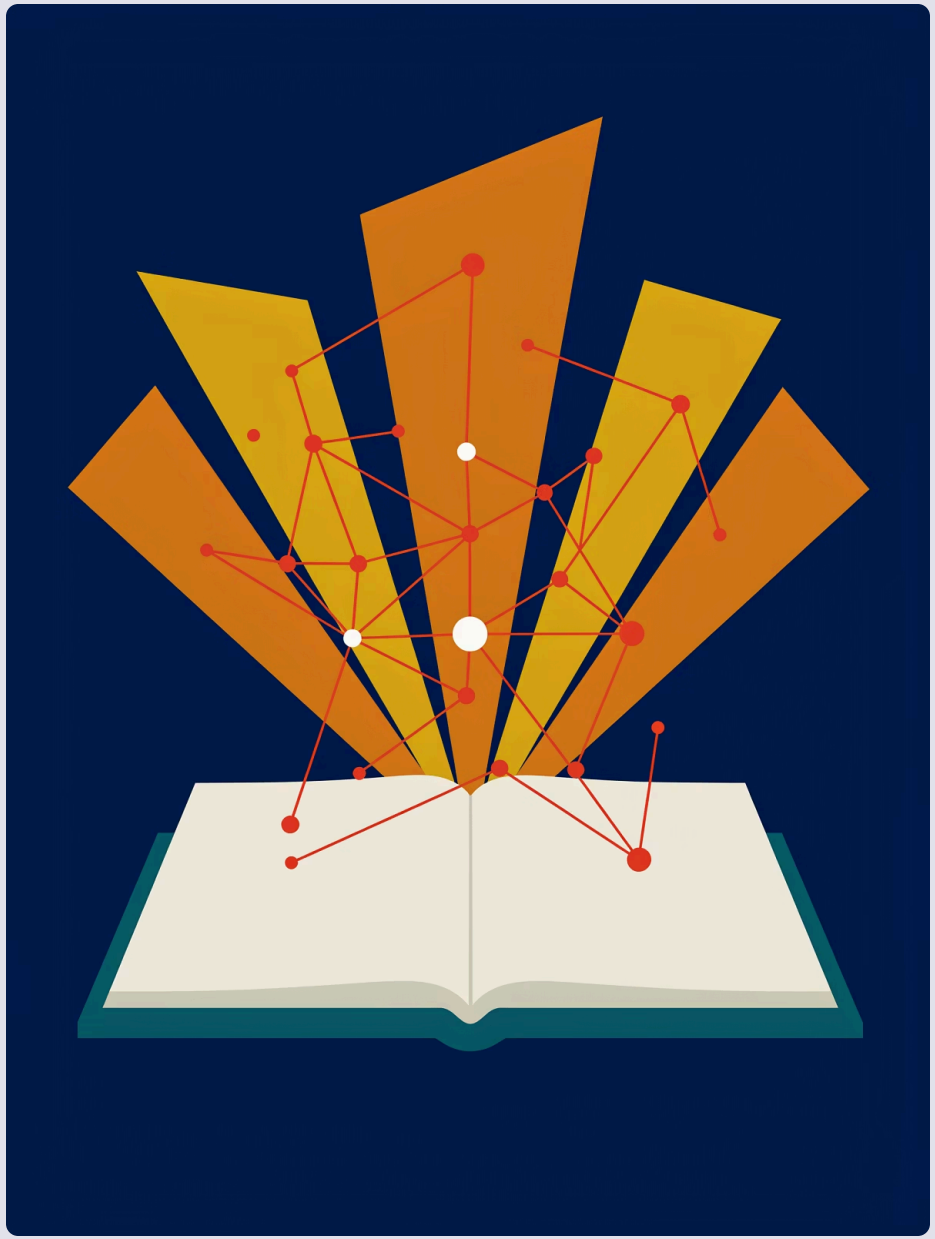
# How a Transformer Works

From reading a sentence to writing a translation

Introduction to Generative AI



# What Is a Transformer?

A system that **reads text, understands it, and produces an output**:

- **A translation**
- **A summary**
- **An answer to a question**



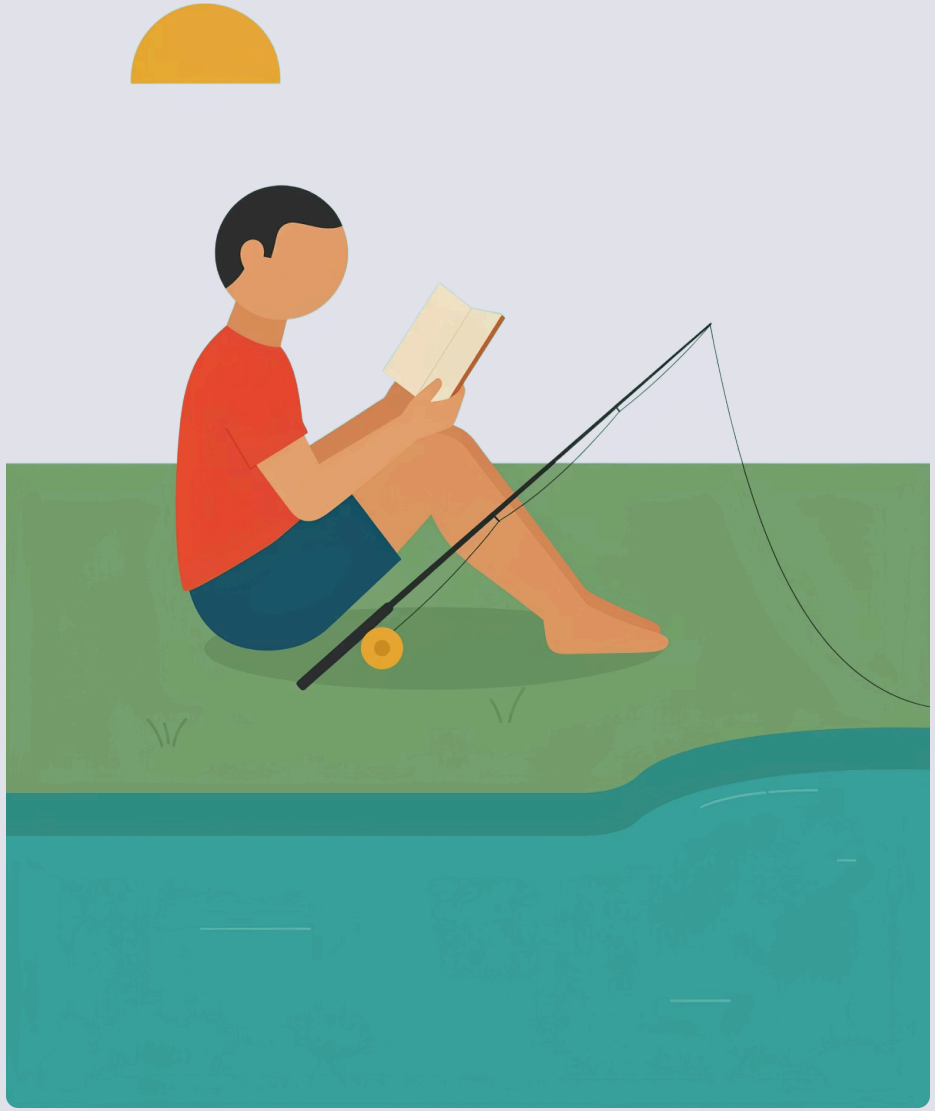
# Think of a Very Attentive Reader

"I went to the river bank to fish."

You instantly know "bank" means the side of a river, not a financial institution.

**Why?** You paid attention to the words around it: "river" and "fish."

A Transformer does exactly the same thing.





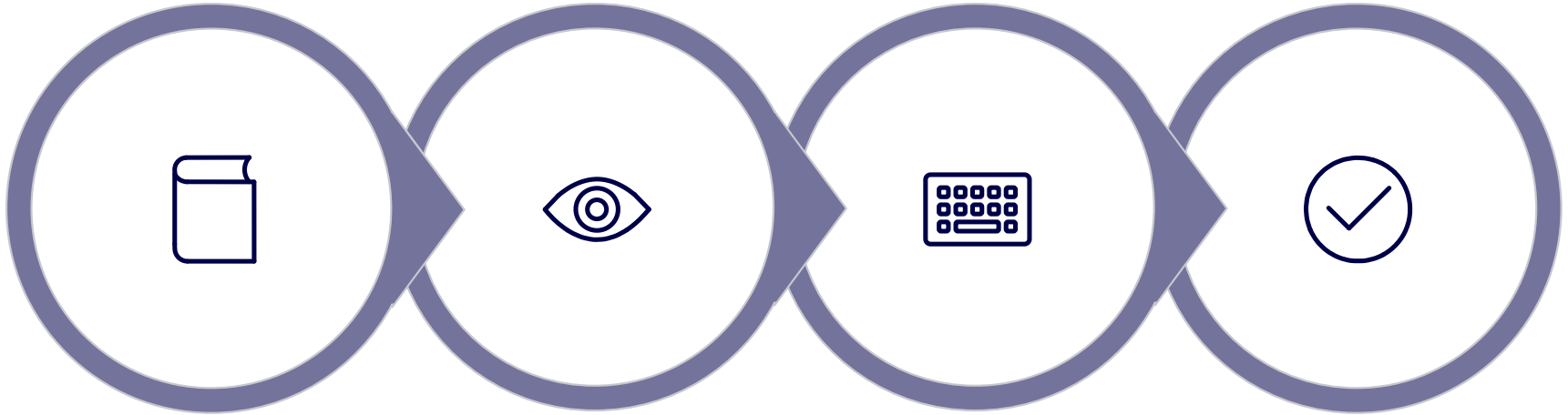
# The Big Picture: A Library Analogy

You ask a librarian to translate a book from English to French. The librarian does two things:

1. **Read and understand the English book completely first. This is the Encoder.**

2. **Write out the French, one word at a time, referring back to the English as needed. This is the Decoder.**

# The Transformer in a Nutshell



Hand Over Book

Encoder Reads

Decoder Writes

French Ready



# Our Example Sentence

**English:**

"The bank can guarantee deposits."

**French:**

"La banque peut garantir les dépôts."

"bank" is tricky: river bank or financial bank? The Transformer must work it out from **context**.

# Step 1: Breaking the Sentence into Tokens

"The bank can guarantee deposits." becomes:

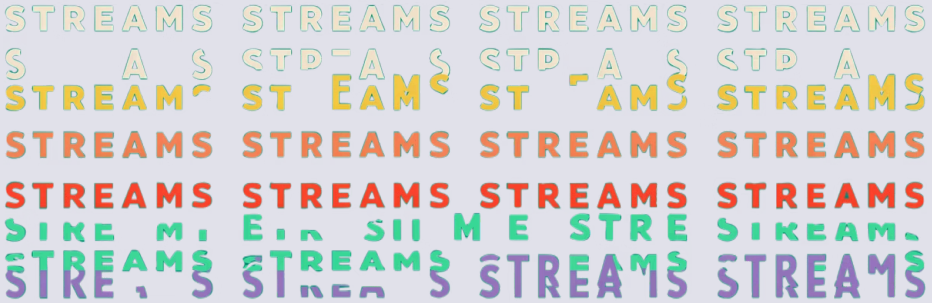
| | | |
|---|---|---|
| **"The"** | **"bank"** | **"can"** |
| **"guarantee"** | **"deposits"** | |

(Real models split into sub-word pieces called tokens. We use whole words to keep it simple.)




# Words Become Numbers

- Each token is converted into a list of numbers
- Computers cannot read words directly, but they can work with numbers
- Think of it as assigning a unique bar code to every word in the dictionary




# Step 2: Giving Each Word a Position Number

The Transformer reads **all words at the same time**. That makes it fast.

But then how does it know "The" comes before "bank"?

# The Position Stamp

Each word also gets a small extra number: "I am in position 1", "position 2", and so on.

01

**"The" → number code + position 1**

02

**"bank" → number code + position 2**

03

**"can" → number code + position 3**

04

**"guarantee" → number code + position 4**

05

**"deposits" → number code + position 5**

Now it knows both **what** each word means and **where** it sits.

# Step 3: The Encoder

The Encoder reads the whole English sentence and understands it deeply, especially the **relationships between words**.

Its key tool: **Attention**.

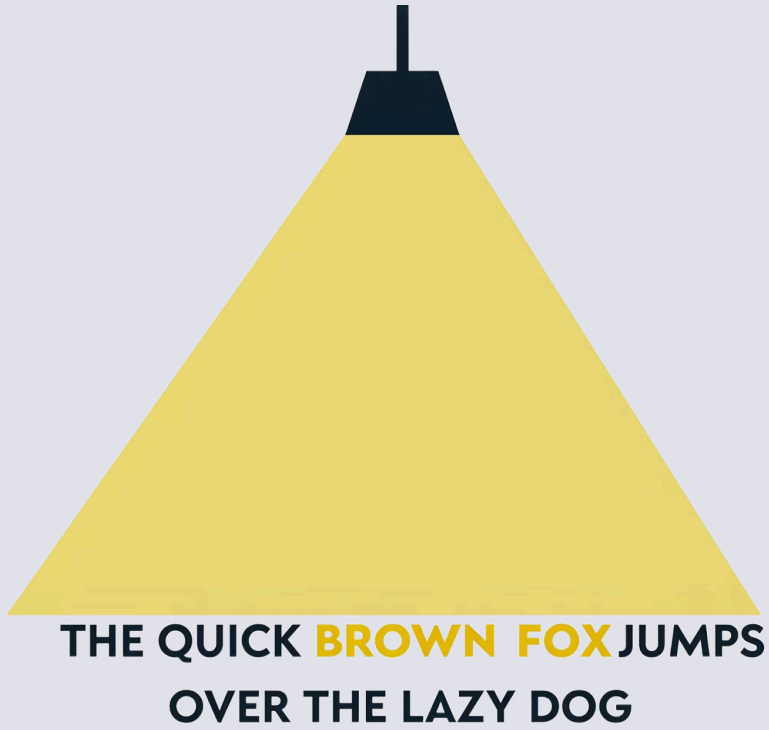


# The Key Idea: Paying Attention

When the Encoder looks at "bank", it asks:

> "Which other words in this sentence help me understand what 'bank' means here?"

It gives every other word a **relevance score**.



# Scoring the Words

| Word | Relevance to "bank" |
| --- | --- |
| The | Low |
| bank | Medium (itself) |
| can | Low |
| **guarantee** | **High** |
| **deposits** | **Very High** |

*Illustrative scores for teaching; real models learn these from data.*



# Self-Attention

"deposits" scores very high, so the Encoder concludes: this is a **financial** bank.

Looking at all words and deciding which matter most is called **Self-Attention**, the heart of the Transformer.

# Multiple Readers: Multi-Head Attention

Instead of one attention check, the Transformer runs **several in parallel**. Each looks for a different relationship:

**Meaning: deposits → financial bank**

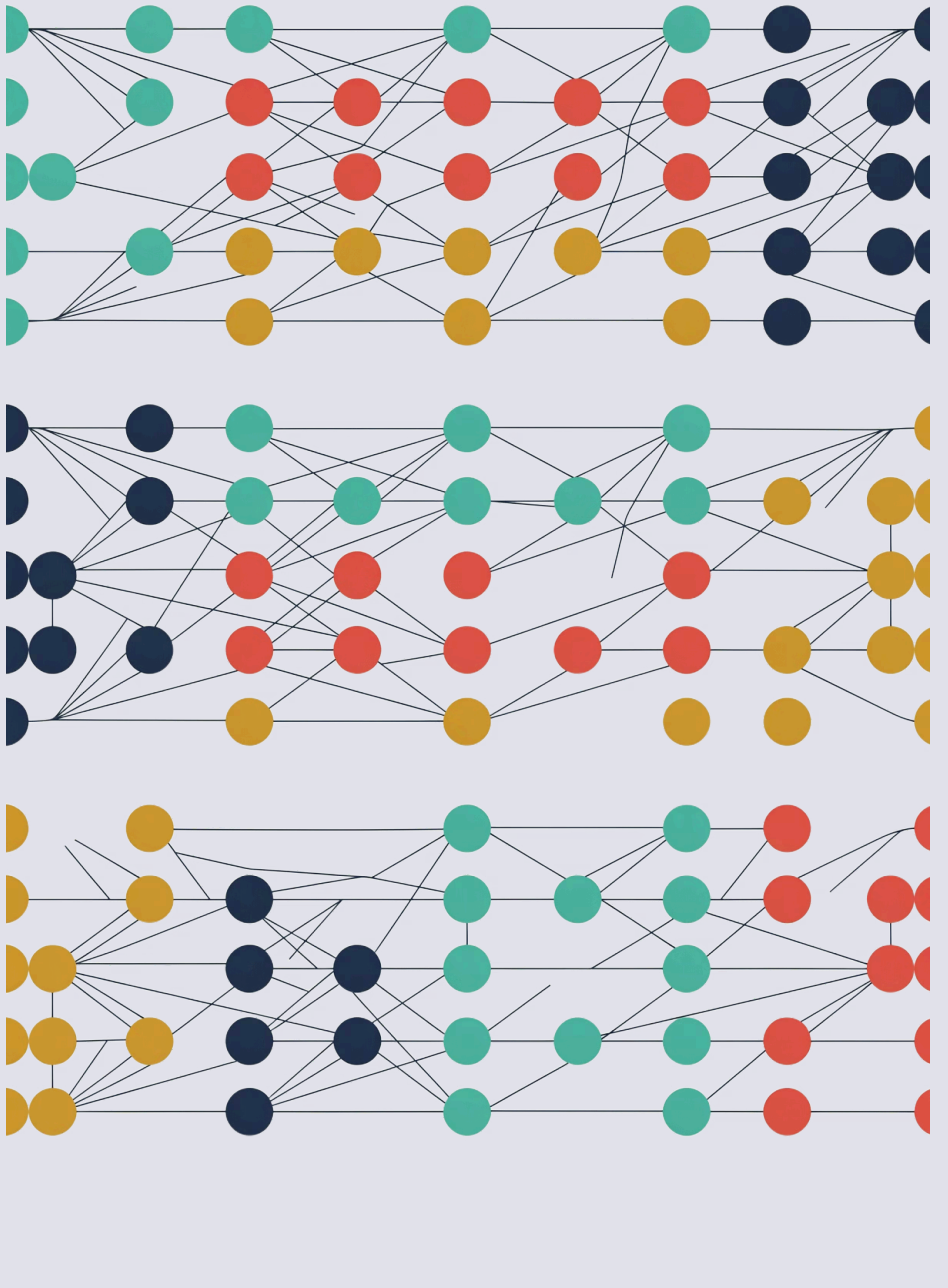
**Grammar: can guarantee → verb phrase**

**Topic: guarantee, deposits, costs → finance**

All passes are combined into one rich understanding.

# Passing Through Multiple Layers

- After attention, each word's meaning is refined by a small calculation
- The whole process (attention + refinement) repeats several times
- The original model used 6 layers
- By the end, every word's representation has absorbed context from all surrounding words



# Step 4: The Decoder

Once the Encoder has understood the English, the Decoder writes the French **one word at a time**.

It does two things for each word:

1. **Looks back at the Encoder (Cross-Attention)**

2. **Remembers what it has already written**



# The Decoder Refers Back to the Encoder

For each French word, it asks: **"Which English word am I translating right now?"**

| English Word | Attention when writing "banque" |
|---|---|
| "The" | low attention |
| **"bank"** | **VERY HIGH attention ← translating this** |
| "can" | low attention |
| "guarantee" | low attention |
| "deposits" | low attention |

This look-back is called **Cross-Attention**.

# The Decoder Remembers What It Wrote

It does not start fresh for each word. Each new word makes sense in the context of the **growing translation**.

01

**Write "La"**

02

**Already have "La" → Write "banque"**

03

**Already have "La banque" → Write "peut"**

04

**Already have "La banque peut" → Write "garantir"**

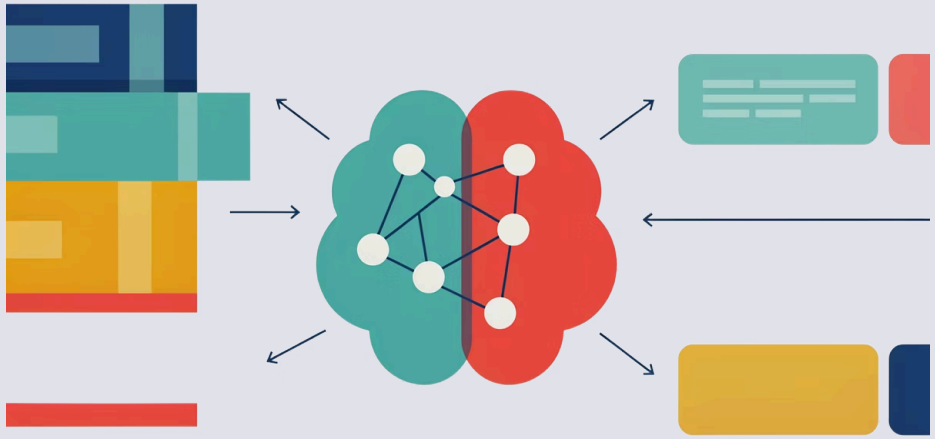
05

**...and so on until the sentence is complete.**

# Step 5: Picking the Final Word

At each step, the Decoder scores **every word in its vocabulary** and picks the highest.

| Word | Score at step 2 (after writing "La") |
|---|---|
| **"banque"** | **0.85 ← highest, so chosen** |
| "rivière" | 0.03 |
| "rive" | 0.02 |
| "garantie" | 0.01 |
| ... (all other French words) | very low |



# How Does It Learn All This?

The Transformer is **not programmed** with grammar or word-meaning rules. It **learns from examples**: millions of sentence pairs like:

**English:**

"The bank can guarantee deposits."

**French:**

"La banque peut garantir les dépôts."

# The Training Loop

For each pair, the model:

1. **Makes a guess at the French translation**
2. **Compares its guess to the correct answer**
3. **Adjusts its internal numbers slightly to do better next time**

After millions of examples, attention scores and word representations become highly accurate, **with no human-written rules**.

# Summary: The Parts in Plain Language (1 of 2)

| Part | What it does |
| --- | --- |
| **Tokenization** | Splits the text into small pieces (tokens) |
| **Embeddings** | Converts each piece into numbers a computer can use |
| **Positional Encoding** | Tells the model the order of the words |
| **Encoder** | Reads and understands the full input sentence |
| **Self-Attention** | Figures out which words relate to which other words |

# Summary: The Parts in Plain Language (2 of 2)

| Part | What it does |
| --- | --- |
| **Multi-Head Attention** | Repeats the check several times, looking for different relationships |
| **Decoder** | Writes the output one word at a time |
| **Cross-Attention** | Looks back at the encoder to know what to translate next |
| **Training** | Learns from millions of examples, no rules hand-coded |

# Why Is the Transformer Better Than Older Methods?

### Before

Machines read one word at a time, left to right, like reading a book very slowly. By the end of a long sentence, they had mostly **forgotten the beginning**.

### Transformer

Reads the **entire sentence at once** and connects any word to any other word directly, however far apart.

Result: more accurate on long, complex sentences, and **much faster to train**.

# Where Is the Transformer Used Today?

The same architecture is behind:



**ChatGPT and Claude: conversational AI assistants**



**Google Translate: automatic translation**



**GitHub Copilot: AI that writes code**



**Speech recognition: spoken words to text**



**Medical tools: reading and summarising clinical notes**

# Conclusion

The Transformer does one powerful thing really well: **paying attention to the right words at the right time**.

Just as a human uses surrounding words to understand "bank", the Transformer uses attention to do the same thing, **at massive scale and speed**.



# Quick Check: Discuss in Pairs

1

**What are the two jobs of the Encoder and the Decoder? Use the librarian analogy.**

2

**Why does the model need position stamps if it reads everything at once?**

3

**What is the difference between self-attention and cross-attention?**

4

**Does anyone write grammar rules into a Transformer? How does it learn instead?**